Aula 31:
Hooks: useEffect e Ciclo de Vida

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

Índice

- INTRODUÇÃO
- CICLO DE VIDA VS SINCRONIZAÇÃO
- ANATOMIA DO HOOK USEEFFECT
- FUNÇÃO DE LIMPEZA (CLEANUP)
- FETCH DE DADOS
- BOAS PRÁTICAS
- ERROS COMUNS
- EXERCÍCIOS

Introdução

EFEITOS COLATERAIS

- Operações que interagem com o mundo exterior fora do escopo da renderização pura do React

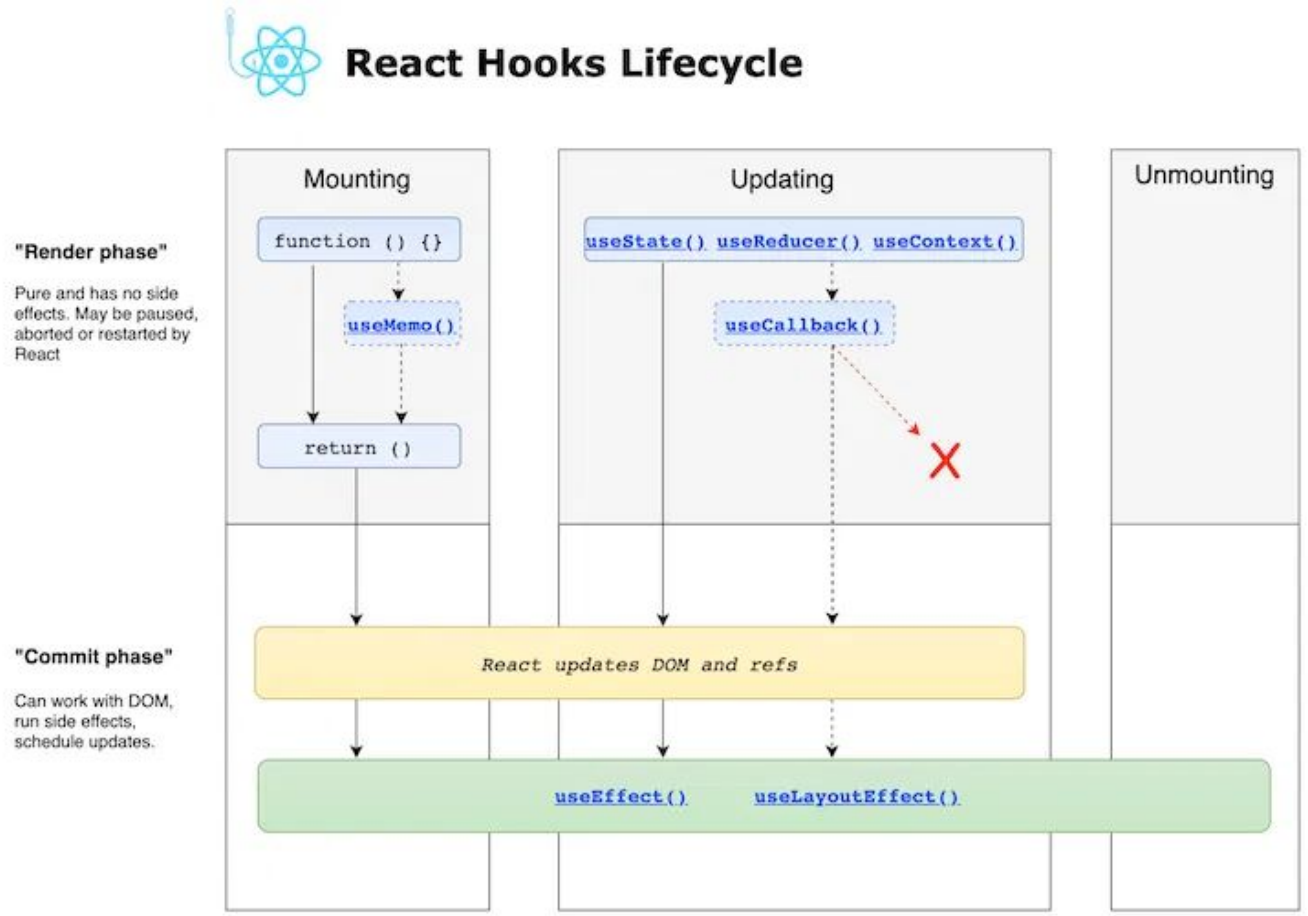
Categoria	Exemplos
Sincronização	APIs, WebSocket, LocalStorage
Manipulação	document.title, setTimeout, setInterval
Assinaturas	Event Listeners (mouse, resize)

Regra: O corpo do componente deve ser PURO

Ciclo de vida vs. Sincronização

FLUXO DE DADOS UNIDIRECIONAL

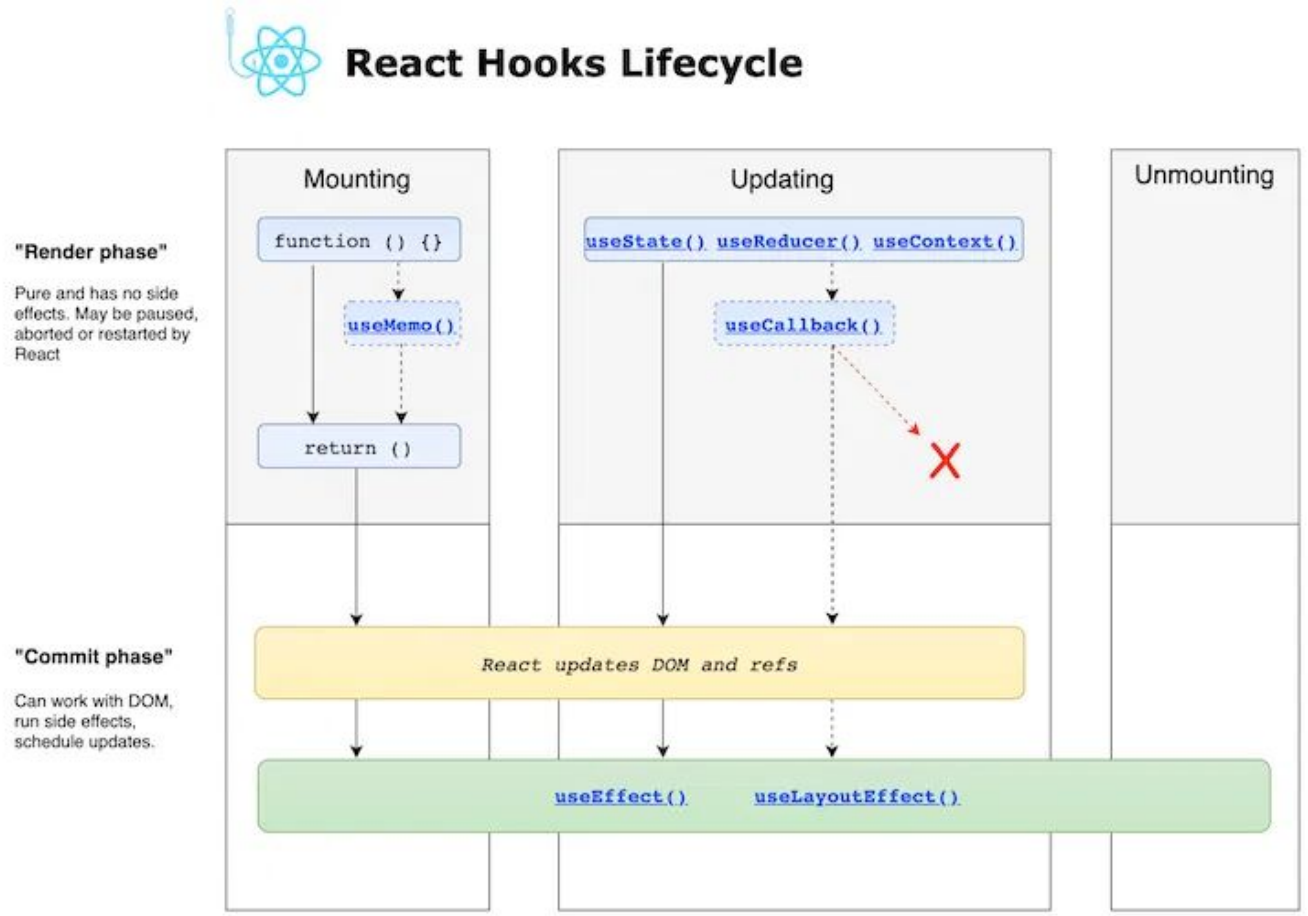
- Mentalidade de Ciclo de Vida
- Focada em eventos temporais:
  - "Quando o componente nasce"
  - "Quando ele muda"
  - "Quando ele morre"



Ciclo de vida vs. Sincronização

FLUXO DE DADOS UNIDIRECIONAL

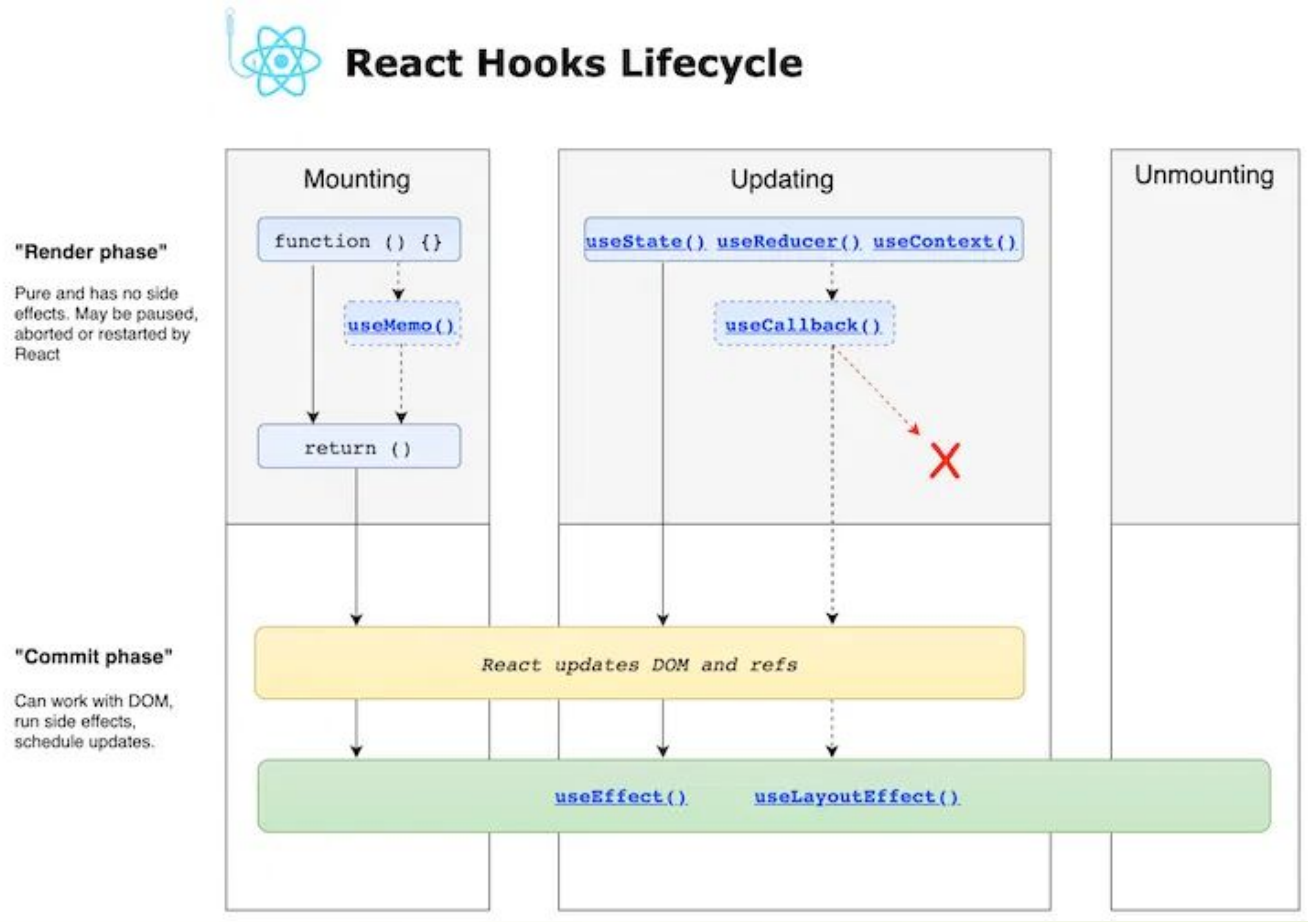
- Mentalidade de Sincronização
- Focada no estado:
  - "Com o que este efeito deve estar sincronizado agora?"



Ciclo de vida vs. Sincronização

FLUXO DE DADOS UNIDIRECIONAL

- A Mudança:
  - O useEffect não é apenas um substituto para métodos de classe
  - É uma forma de manter o sistema externo em sincronia com as props e o estado atuais



Anatomia do Hook UseEffect

- O Hook useEffect permite que você execute efeitos colaterais em seus componentes
- Alguns exemplos de efeitos colaterais são: buscar dados, atualizar o DOM diretamente e temporizadores
- useEffect aceita dois argumentos
- O segundo argumento é opcional:
  - useEffect(<função>, <dependência>)

Anatomia do Hook UseEffect

```
useEffect(() ⇒ {
  // 1. O código do Efeito (Side Effect)
  console.log('O efeito rodou!');

  return () ⇒ {
    // 2. Opcional: Função de Limpeza (Cleanup)
    console.log('O efeito foi limpo.');
  };
}, [dependências]); // 3. Opcional: Array de Dependências
```

1. Corpo do Efeito

Onde a lógica principal é executada. Roda após a renderização e a atualização do DOM.

2. Função de Limpeza

Opcional. Executada antes da re-execução do efeito e quando o componente é desmontado.

3. Dependências

Lista de valores reativos. O efeito só re-executa se algum desses valores mudar.

Anatomia do Hook UseEffect

ARRAY DE DEPENDÊNCIAS

- O segundo argumento do useEffect define quando o efeito deve ser re-sincronizado com o estado da aplicação

Array	Exemplos
`[]` (Vazio)	Executa apenas uma vez após a montagem inicial e limpa na desmontagem
`[dep1, dep2]`	Executa na montagem e sempre que qualquer valor no array mudar.
Omitido	Executa após cada renderização do componente

Anatomia do Hook UseEffect

ARRAY DE DEPENDÊNCIAS

- O segundo argumento do useEffect define quando o efeito deve ser re-sincronizado com o estado da aplicação

Array	Exemplos
`[]` (Vazio)	Fetch inicial de dados, configuração de `setInterval`.
`[dep1, dep2]`	Sincronizar com props ou estados específicos (ex: busca por ID).
Omitido	Raramente usado. **Risco alto de loops infinitos.**

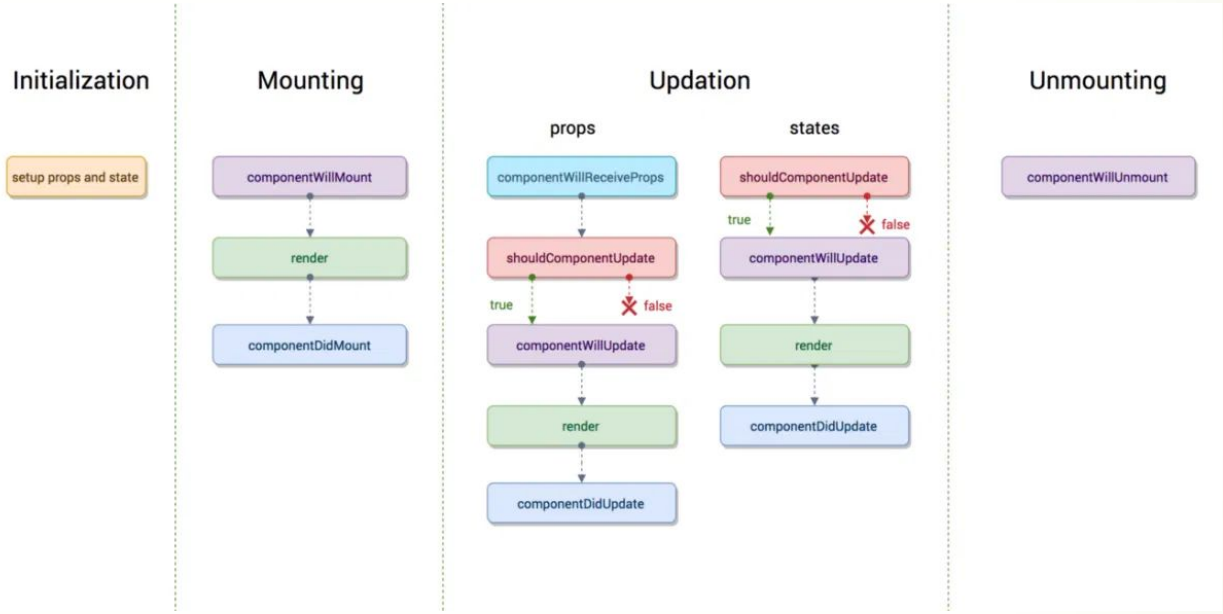
Exemplo prático

ENTENDENDO O USEEFFECT

- Exemplo realizado em conjunto
- Será utilizado o exemplo do site W3Schools

Função de Limpeza (Cleanup)

- Alguns efeitos requerem limpeza para reduzir vazamentos de memória
- Tempo limite, assinaturas, ouvintes de eventos e outros efeitos que não são necessários devem ser descartados
- Fazemos isso incluindo uma função de retorno no final do gancho useEffect



Função de Limpeza (Cleanup)

GARANTINDO A INTEGRIDADE DO SISTEMA

- **Quando ocorre**: Antes da re-execução do efeito e na desmontagem do componente
- **Assinaturas**: Remova addEventListener para evitar múltiplos registros e comportamentos erráticos
- **Timers**: Limpe setTimeout e setInterval para evitar execuções em componentes que não existem mais

Função de Limpeza (Cleanup)

GARANTINDO A INTEGRIDADE DO SISTEMA

- **Conexões**: Feche WebSockets ou cancele subscrições de APIs para economizar recursos
- Sempre que um efeito "assina" algo, ele deve "desassinar" na função de limpeza para evitar Memory Leaks

Exemplo prático

- Fazer as atividades de Cleanup em conjunto
- As atividades de Cleanup são do site W3Schools

Fetch de dados

- Problema:
  - Race Conditions
- Ocorre quando múltiplas requisições assíncronas são disparadas e retornam em ordem inesperada
- Pode causar a exibição de dados obsoletos ou erros em componentes já desmontados

Fetch de dados

- Solução:
  - AbortController
- API nativa do navegador que permite cancelar uma ou mais requisições Web (Fetch)
- Implementação:
  - Criar o controller no início do efeito e chamar abort() na função de limpeza

Exemplo Prático

FETCH DE DADOS

- Objetivo: Criar um buscador de usuários que lida com requisições assíncronas de forma segura

1. Interface e Estado

- Crie um input controlado para o termo de busca.
- Gerencie estados para users, loading e error.
- Use a API: https://jsonplaceholder.typicode.com/users

Exemplo Prático

FETCH DE DADOS

- Objetivo: Criar um buscador de usuários que lida com requisições assíncronas de forma segura

2. Lógica e efeito

- Dispare o fetch sempre que o termo de busca mudar.
- Implemente o AbortController para cancelar buscas pendentes.
- Trate o erro AbortError para não exibir alertas desnecessários.

Boas Práticas

1. Separação de Interesses

- Use múltiplos useEffect para lógicas independentes
- Isso torna o código mais legível e fácil de testar

2. Dependências Reais

- Sempre inclua todas as variáveis reativas usadas no efeito
- Confie no ESLint para garantir a integridade

Boas Práticas

3. Funções Estáticas

➢ Mova funções que não dependem de props ou state para fora do componente para evitar re-execuções

Erros Comuns

1. Loops Infinitos

- Atualizar um estado que está no array de dependências sem uma condição de parada clara

2. Stale Clousures

- Omitir dependências, fazendo com que o efeito utilize valores antigos de props ou estados

Erros Comuns

3. Esquecer o Cleanup

- Não limpar timers ou event listeners, resultando em vazamentos de memória e bugs silenciosos

Exercícios

Validação de Formulário com Debounce

- Crie um formulário simples com um campo de email

1. Use o useState para o valor do email e para o estado de validação (isValid).

2. Use o useEffect para validar o email (ex: verificar se contém '@' e '.') após o usuário parar de digitar por 500ms (técnica de debounce).

3. CRÍTICO: Use setTimeout para o debounce e implemente o cleanup para cancelar o timer se o usuário digitar novamente antes dos 500ms. O useEffect deve ser re-executado sempre que o valor do email mudar.

Exercício: Gerenciador de Tarefas

- Ao longo deste curso, vamos desenvolver progressivamente uma aplicação React completa, aplicando os conceitos estudados em cada aula
- O projeto será construído de forma incremental: A cada novo conteúdo, novas funcionalidades deverão ser adicionadas
- Ao final do curso, você teremos:
  - Uma aplicação React funcional
  - Um projeto organizado, testado e otimizado
  - Um deploy público em ambiente de produção

Exercício: Gerenciador de Tarefas

- O sistema consiste em uma aplicação web SPA (Single Page Application) para gerenciamento de projetos e tarefas, permitindo que o usuário:
  - Visualize projetos cadastrados
  - Acesse tarefas associadas a cada projeto
  - Crie, edite, conclua e remova tarefas
  - Navegue entre diferentes páginas da aplicação
  - Tenha seus dados persistidos localmente

Exercício: Gerenciador de Tarefas

- A aplicação deverá conter, no mínimo, os seguintes componentes:
  - App
  - Header
  - ProjectList
  - TaskList
  - TaskItem
  - TaskForm

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: UseEffect**
  - Usar useEffect com array de dependências vazio [] para ler do localStorage ao abrir o app
  - Usar useEffect com dependência [tasks] para salvar no localStorage toda vez que uma tarefa for adicionada ou removida
  - Extra: Alterar o document.title dinamicamente: "TaskMaster (3 pendentes)"
- Funcionalidade: Salvar as tarefas no Navegador (LocalStorage) para não sumir ao dar F5
- Entrega: É possível adicionar tarefas, fechar a aba, abrir de novo e as tarefas continuam lá